Android Application Files — A Reference for FBO 3.0

What lives where, in what format, and how to enumerate it

2026-06-07

1. The mental model

An installed Android application exists in two completely separate places on disk, owned by different parts of the system and governed by different lifecycle rules. Conflating them is the single most common source of confusion when reasoning about file-level optimizations like FBO 3.0.

The distribution side — the files that came in with the APK install, the ART-compiled artifacts derived from them, and any extracted native libraries. These are read-only after installation, regenerated only on app update or system update, and dominate the I/O of cold app launch.

The runtime side — everything the app writes during normal operation: preferences, SQLite databases, downloaded content, caches, JIT compilation artifacts. These are read-write and their layout is dictated by the app itself.

For an app like Antutu (a benchmark), the cold-launch hot set is almost entirely on the distribution side. The benchmark scores reflect how quickly bytecode and native libraries can be loaded from storage — operations that exclusively touch `/data/app`. Pinning the distribution side is what FBO 3.0 wants. The data side is out of scope for this document.

---

2. The distribution side: `/data/app`

Every installed third-party application has its distribution files placed under `/data/app`, in a per-install directory whose path contains two randomized components inserted by `PackageInstaller`. The directory structure is otherwise completely regular.

2.1 The directory layout

```
/data/app/
└── <random-hash-1>/                          # randomized per install
    └── <package-name>-<random-hash-2>/       # randomized per install
        ├── base.apk
        ├── split_config.<abi>.apk            # zero or more
        ├── split_config.<density>.apk        # zero or more
        ├── split_config.<locale>.apk         # zero or more
        ├── lib/                              # may or may not exist
        │   └── <isa>/                        # arm64, arm, x86_64, x86
        │       ├── libfoo.so
        │       └── libbar.so
        └── oat/
            └── <isa>/
                ├── base.odex
                ├── base.vdex
                └── base.art
```

The two random hash components — let's call them `H1` and `H2` — are generated at install time. They change on every reinstall (including reinstalls triggered by app update). They exist to prevent other applications from constructing predictable paths into another app's install directory; this is part of the broader Android sandbox hardening that took shape from Android 8 onwards.

For your tooling, the critical implication is: **never hardcode paths under `/data/app`**. Always resolve them through `PackageManager`. The hashes are not predictable, but the structure beneath them is.

2.2 The files, in detail

`base.apk`

The primary APK. Despite the `.apk` extension, this is a standard ZIP archive. You can confirm this with `unzip -l base.apk` or `file base.apk`. Its contents follow a well-defined layout:

```
base.apk (ZIP)
├── AndroidManifest.xml         # binary XML, not text
├── classes.dex                 # primary Dalvik bytecode
├── classes2.dex                # secondary DEX files
├── classes3.dex                # for apps that overflow the
├── ...                         # single-DEX limit
├── resources.arsc              # compiled resource table
├── res/
│   ├── drawable-xxhdpi/
│   ├── layout/
│   └── ...
├── assets/                     # raw, uncompiled developer assets
├── lib/                        # only if extractNativeLibs=false
│   └── <abi>/
│       └── *.so                # page-aligned, uncompressed
└── META-INF/
    ├── MANIFEST.MF
    ├── CERT.SF
    └── CERT.RSA                # the signing certificate chain
```

What gets read at cold launch:

- `AndroidManifest.xml` is parsed once by `PackageManager` at install time, then again briefly at launch.
- `classes*.dex` are mmap'd by ART. On a fully AOT-compiled app, the `.dex` reads are minimal — most of the actual code execution comes from the `.odex` file in `oat/`. On a not-yet-compiled or interpretation-only app, the `.dex` reads dominate.
- `resources.arsc` is mmap'd and frequently accessed; resource lookups touch it constantly during UI inflation.
- Anything under `res/` is read on demand when the corresponding drawable / layout is first referenced.
- Anything under `lib/` is mmap'd if and when the app calls `System.loadLibrary`. For Antutu specifically, this happens early.

`split_config.*.apk`

Android App Bundles (the modern distribution format since 2018) allow Play Store to deliver only the splits that match a given device. A device running on `arm64-v8a` with `xxhdpi` density and English locale might receive:

- `base.apk` — code and resources shared across all configurations
- `split_config.arm64_v8a.apk` — ABI-specific native code
- `split_config.xxhdpi.apk` — density-specific drawables
- `split_config.en.apk` — locale-specific strings

Each split is an APK in its own right (same ZIP structure), and ART loads them as a set. The Antutu install on your dev device may or may not have splits depending on how it was downloaded.

For FBO purposes, splits are pinned the same way as `base.apk`. The enumeration step from `pm path` returns the entire set in one call.

`oat/<isa>/base.odex`

This is where most of the cold-launch performance lives.

When an app is installed (or when `dex2oat` runs in the background due to "Optimizing apps" prompts after an OTA), Android compiles the DEX bytecode into native machine code for the device's ISA. The result is an ELF binary placed at `oat/<isa>/base.odex`. ART maps this file directly into the app's address space at launch and executes from it.

The `<isa>` directory uses the short instruction set name: `arm64` for 64-bit ARM, `arm` for 32-bit ARM, `x86_64`, `x86`. This is **not** the same as the ABI name used for native libs (where it's `arm64-v8a`, `armeabi-v7a`, etc.) — a small inconsistency that catches everyone the first time.

The compilation level varies. Possible states for any given `.odex`:

- `speed` — fully AOT compiled. Fastest startup, biggest file.
- `speed-profile` — profile-guided AOT, only hot methods compiled. Smaller, still fast for the methods that matter.
- `quicken` — minimal compilation, just bytecode quickening.
- `verify` — verified only, no compilation.

Modern Android leans heavily on `speed-profile`: the system collects a profile of which methods the app actually executes, then re-compiles during idle/charging hours with that profile as input. This means the `.odex` file may be **rewritten** weeks after the app was installed, without any user-visible action. This is a wrinkle you will need to handle if you keep pins valid long-term — file mtime or inode change on the `.odex` is your signal that the previous pin record is stale.

`oat/<isa>/base.vdex`

Verified DEX. Contains the original DEX bytecode plus the verifier's metadata. ART uses this as a fallback (for methods not present in `.odex`) and for verification. Smaller than the raw `.dex` files would be, mmap'd at launch.

`oat/<isa>/base.art`

The boot image / class preloader. A pre-cooked memory image containing pre-resolved class objects, string-interning tables, and other ART runtime structures. Loaded very early in the app launch sequence — before any application code runs — and provides much of the warm-start speedup over a cold ART boot.

If you had to pick a single file whose pinning gives the most cold-start benefit per pinned byte, `.art` is the candidate. It's small, it's the first thing read, and its access pattern is read-once-sequential which is the worst case for unpinned flash on a phone that just woke up.

`lib/<isa>/*.so`

Native libraries, extracted to disk at install time, **if** the app manifest has `android:extractNativeLibs="true"`.

A naming quirk to be aware of: inside the APK the libs live under `lib/<abi>/` using the full Android ABI name (`arm64-v8a`, `armeabi-v7a`, `x86_64`, `x86`). When extracted to disk under `/data/app/.../lib/`, the directory is renamed to the **short ISA** form (`arm64`, `arm`, `x86_64`, `x86`) — the same naming used for `oat/<isa>/`. The mapping is `arm64-v8a -> arm64`, `armeabi-v7a -> arm`, others unchanged. So a file that ships at `lib/arm64-v8a/libfoo.so` inside `base.apk` ends up as `<install-dir>/lib/arm64/libfoo.so` on disk after extraction.

Since Android 6, the default has been to *not* extract — instead the `.so` files live inside `base.apk` (or inside a `split_config` APK), page-aligned and stored uncompressed, and ART mmap's them directly out of the APK at the right offsets. This is more efficient (no duplicated data on disk) and is the modern default.

For the FBO 3.0 enumeration, you handle both cases:

- If `lib/<isa>/` exists under the install dir: pin those `.so` files as separate entries.
- If it does not exist: the libs are inside `base.apk` (or splits), so pinning the APK covers them.

`dumpsys package <package>` reports `legacyNativeLibraryDir` and `primaryCpuAbi` from which you can

derive the exact native lib directory if extraction happened.

2.3 What's the total surface area?

For a typical mid-sized app on a modern device:

Component	Typical size
`base.apk`	20 – 200 MB
Splits (total)	5 – 50 MB
`base.odex`	similar size to `classes*.dex` in the APK, often 10 – 100 MB
`base.vdex`	5 – 50 MB
`base.art`	1 – 10 MB
Extracted libs (if any)	5 – 100 MB

Antutu specifically, given its size and the embedded benchmark payloads, sits at the higher end of these ranges. A reasonable upper bound for the full distribution-side footprint of a single benchmark-sized app: 300–500 MB.

For an OEM building an FBO 3.0 feature, the UFS pin region budget needs to be sized with this in mind. A pin region of, say, 1 GB can fit two to three apps' worth of distribution files comfortably; an LRU or LFU eviction policy decides which apps remain pinned as the user runs different things.

---

3. The runtime side: `/data/data` (briefly)

The runtime data tree, mentioned for completeness and to make clear why you exclude it from FBO scope:

```
/data/data/<package-name>/
├── files/              # arbitrary files the app writes
├── cache/              # caches; the system may clear on pressure
├── databases/          # SQLite DBs
├── shared_prefs/       # key-value XML preferences
├── code_cache/         # JIT artifacts, JIT-PGO profiles
├── no_backup/          # opt-out of cloud backup
└── app_<name>/         # arbitrary sub-directories the app creates
```

For benchmark and game-perf use cases, the data tree is overwhelmingly write-heavy or cold-read. Pinning it would consume budget without yielding the perf signal that benchmarks measure. Hence the scope decision in FBO 3.0 to ignore it.

For other product use cases (browser cache, chat app message database) the data tree matters a lot — but that's a different feature with different lifecycle rules and is explicitly out of scope here.

---

4. The other relevant tree: `/data/dalvik-cache`

On some Android versions and for some apps (particularly system apps under `/system/app` and `/system/priv-app`), the ART artifacts live not next to the APK in `oat/`, but in a central `/data/dalvik-cache` tree:

```
/data/dalvik-cache/
└── <isa>/
    ├── system@app@<AppName>@<AppName>.apk@classes.dex
    ├── system@app@<AppName>@<AppName>.apk@classes.odex
```

```
    ├── system@app@<AppName>@<AppName>.apk@classes.vdex
    └── system@app@<AppName>@<AppName>.apk@classes.art
```

The filename encoding takes the original APK path and replaces `/` with `@`. This is legacy from the original Dalvik VM. Modern third-party apps (Play Store installs) keep their OAT artifacts under `/data/app/.../oat/`, but you may still see `/data/dalvik-cache` entries for boot-class-path artifacts (the framework's own OAT files that live under the system server).

For FBO scope, you generally do not pin from `/data/dalvik-cache` unless you're targeting system apps. For the Antutu case, all the relevant OAT is under `/data/app/.../oat/`.

---

5. Enumeration: how to get the file list

There are several layers of API for resolving "what are the files for this app?". From most-to-least-shell-friendly:

5.1 `pm path` — the shell primitive

```
pm path com.antutu.ABenchMark
```

Output:

```
package:/data/app/<H1>/com.antutu.ABenchMark-<H2>/base.apk
package:/data/app/<H1>/com.antutu.ABenchMark-<H2>/split_config.arm64_v8a.apk
package:/data/app/<H1>/com.antutu.ABenchMark-<H2>/split_config.xxhdpi.apk
```

One line per APK (base + each installed split), prefixed with `package:`. Strip the prefix and you have absolute paths.

This call goes through `PackageManagerService` and uses the same internal data structures Android itself uses to find the files at launch. You will not get a wrong answer from it.

5.2 `dumpsys package` — the verbose alternative

```
dumpsys package com.antutu.ABenchMark
```

A much larger output, but contains everything: install location, data dir, native lib dir, primary ABI, version codes, granted permissions. The fields of interest for FBO are:

```
codePath=/data/app/<H1>/com.antutu.ABenchMark-<H2>
resourcePath=/data/app/<H1>/com.antutu.ABenchMark-<H2>
legacyNativeLibraryDir=/data/app/<H1>/com.antutu.ABenchMark-<H2>/lib
primaryCpuAbi=arm64-v8a
```

`codePath` is the install directory. Append `/oat/<isa>` to get to the OAT artifacts. `legacyNativeLibraryDir` is set whether or not extraction occurred; check whether the directory actually exists and is non-empty.

5.3 PackageManager API — when your code runs as an Android app

```
val pm = context.packageManager
val ai: ApplicationInfo = pm.getApplicationInfo("com.antutu.ABenchMark", 0)

val files = mutableListOf<String>()
files += ai.sourceDir                           // base.apk
ai.splitSourceDirs?.let { files += it }         // splits
files += File(ai.nativeLibraryDir).listFiles()
```

```
            ?.map { it.absolutePath } ?: emptyList()

        val installDir = File(ai.sourceDir).parent
        val abi = Build.SUPPORTED_ABIS[0]               // e.g. "arm64-v8a"
        val isa = when (abi) {
            "arm64-v8a"   -> "arm64"
            "armeabi-v7a" -> "arm"
            "x86_64"      -> "x86_64"
            "x86"         -> "x86"
            else          -> abi
        }
        files += File("$installDir/oat/$isa").listFiles()
            ?.map { it.absolutePath } ?: emptyList()
```

This is the form your eventual controller daemon will use if it ships as a privileged Android service rather than a CLI tool.

5.4 A complete shell-side enumeration script

For your prototype, this is sufficient:

```
#!/system/bin/sh
PKG=$1
[ -z "$PKG" ] && { echo "usage: $0 <package>"; exit 1; }

ABI=$(getprop ro.product.cpu.abi)
case "$ABI" in
    arm64-v8a)   ISA=arm64 ;;
    armeabi-v7a) ISA=arm   ;;
    x86_64)      ISA=x86_64 ;;
    x86)         ISA=x86   ;;
    *)           ISA=$ABI  ;;
esac

# APKs (base + splits)
APKS=$(pm path "$PKG" 2>/dev/null | sed 's/^package://')
[ -z "$APKS" ] && { echo "package not installed: $PKG"; exit 1; }

# Install directory derived from first APK path
DIR=$(dirname "$(echo "$APKS" | head -1)")

echo "$APKS"
ls "$DIR/oat/$ISA"/* 2>/dev/null     # base.odex, base.vdex, base.art
ls "$DIR/lib/$ISA"/* 2>/dev/null     # only if extracted (uses ISA, not ABI)
```

This is the complete static pinnable surface for one package. For Antutu, expect a list of 8 to 20 files.

---

6. The complete FBO pin sequence, per file

Given a single file path resolved by the enumeration above, the sequence to make it pinned and durable is:

```
int fd = open(path, O_RDONLY);

// Step 1: prevent F2FS GC from relocating this file's blocks.
// Without this, the FIEMAP we take next can become stale at any
// time the cleaner runs.
int pin = 1;
ioctl(fd, F2FS_IOC_SET_PIN_FILE, &pin);

// Step 2: get the (logical_offset -> physical_LBA, length) map.
struct fiemap fm = {
    .fm_start        = 0,
    .fm_length       = ~0ULL,
```

```
    .fm_flags        = FIEMAP_FLAG_SYNC,
    .fm_extent_count = N,
};
// ... allocate room for N fiemap_extent entries after fm ...
ioctl(fd, FS_IOC_FIEMAP, &fm);

// Step 3: hand the physical LBA ranges to the UFS pin command.
// Interface varies by vendor — sysfs node, scsi passthrough, etc.
for (each extent in fm.fm_extents) {
    ufs_vendor_pin(extent.fe_physical, extent.fe_length);
}

// Step 4: record what was pinned, so unpin can reverse it later.
record_pin(package, path, inode, mtime, extents);

close(fd);
```

The unpin path reverses the same sequence: walk recorded extents, issue the vendor unpin command, optionally clear the F2FS pin if no other consumer needs it.

---

7. Lifecycle events that matter

A pinned-files registry has to be aware of the events that invalidate its records. These are all observable from a service listening to `PackageManager` broadcasts and from filesystem-level metadata checks.

Event	Mechanism to observe	Action
App installed	`ACTION_PACKAGE_ADDED`	Optional: pin if policy says so
App updated	`ACTION_PACKAGE_REPLACED`	Unpin old extents, re-enumerate, re-pin
App removed	`ACTION_PACKAGE_REMOVED`	Unpin extents
OAT regenerated by `dex2oat`	inode/mtime change on `.odex`	Re-FIEMAP and re-pin OAT subset
System OTA	reboot + `dex2oat` runs for all apps	Re-pin everything tracked
Manual `pm clear-package`	`ACTION_PACKAGE_DATA_CLEARED`	No action (only affects data dir)
F2FS GC moves blocks	should not happen if `F2FS_IOC_SET_PIN_FILE` is set	If somehow it does: detect via stale-extent reads, re-FIEMAP

The OAT-regeneration case is the easy one to forget. Android runs `dex2oat` opportunistically based on collected profiles, on charge, on idle, after OTAs, and at install. Your records need an integrity check before being trusted — comparing recorded inode and mtime against the current file's metadata is sufficient.

---

8. Verification on a development device

Before writing any pinning code, run these on the dev target to confirm everything lines up with this document. Substitute any installed third-party app for `<pkg>`.

```
# Pick a target
PKG=com.antutu.ABenchMark
pm list packages | grep antutu     # confirm install name

# Resolve the file set
pm path $PKG
DIR=$(dirname $(pm path $PKG | head -1 | cut -d: -f2))
```

```
echo $DIR
ls $DIR
ls $DIR/oat/*
ls $DIR/lib/* 2>/dev/null || echo "(no extracted libs — they're inside the APK)"

# Pick one file and confirm we can FIEMAP it
F=$DIR/base.apk
filefrag -e $F | head
# Or, more specifically, use the FIEMAP ioctl from a small C program

# Confirm F2FS pin is available on this kernel
ls /sys/fs/f2fs/                  # should list one entry per F2FS mount
grep -i pin /proc/$(pidof <something on f2fs>)/mountinfo
# Try the ioctl from a small program; success indicates the kernel supports it

# Strace an app launch to see what it actually opens
am force-stop $PKG
strace -f -e openat -o /sdcard/antutu_strace.log am start -n $PKG/<MainActivity>
# Then analyze: which paths matter, what fraction are under /data/app vs /data/data
```

The strace output is the empirical confirmation that the distribution-side enumeration covers what actually matters. For a benchmark like Antutu, expect the overwhelming majority of `openat` calls during launch to be on files under the install directory and on framework files under `/system` (which you typically don't pin per-app).

---

9. Implications for FBO 3.0 architecture

Restating the design choices that fall out of the above, for the record:

The pinnable file set is statically known per installed app. It is enumerable in milliseconds via `pm path` + a directory listing. No runtime observation of `open()` calls is required to discover it.

The set is stable between app launches. Inodes and physical extents do not change as long as F2FS GC is prevented from moving the files (which `F2FS_IOC_SET_PIN_FILE` ensures). Therefore the pin can be applied at any time before the app is launched — at install, on user request, on predictive trigger — and remains effective.

Invalidation is event-driven and small in scope. Three triggers (package replaced, package removed, OAT regenerated) cover every real-world reason a pin record becomes stale. All three are observable without polling.

No `open()` interception is required for the static-file pin use case. The fanotify / kernel-hook design is correct only for use cases where the relevant file set is dynamic (e.g., apps that download gigabytes of content at first launch, where the downloaded files are themselves the perf-critical reads). For benchmark and standard-app acceleration, the static enumeration is complete.

Pin region budget is the real product constraint, not file discovery. UFS pin region size is finite (typically tens of MB to a few GB depending on device). The interesting engineering work is the eviction policy: which app's files stay pinned when the user installs the eighth game in a row. This is a cache-management problem, not an I/O-interception problem.

---

10. Summary

The Android application file layout, for the purposes of FBO 3.0, is:

- **Distribution files** live under `/data/app/<H1>/<package>-<H2>/` with predictable structure but randomized hashes.
- The hot set is `base.apk`, `split_config.*.apk`, `oat/<isa>/*`, and optionally `lib/<isa>/*.so` if extracted (note: on-disk uses the short ISA form, e.g. `arm64`, not the APK-internal `arm64-v8a`).

- The set is enumerable via `pm path` and one directory listing.
- The set is stable between launches; invalidated only by app update, uninstall, or OAT regeneration.
- F2FS pinning before FIEMAP is required to keep the pin durable against background GC.
- No syscall interception is needed for this use case; pin is a proactive, persistent operation on the storage device.

The next concrete step, on the rooted dev target, is to run the enumeration script in section 5.4 against an installed app, then a small C program that performs the FIEMAP step (without the UFS pin command for now) and prints the resulting extent list. Once the extent list looks correct, the only remaining variable is the vendor UFS pin interface, which is OEM-specific and outside the scope of this document.

---

11. Reference implementation

This section contains the complete set of working programs and scripts for FBO 3.0. There is one C program with five subcommands, plus two optional shell helpers. Together they cover both the production pin/unpin path and the validation/analysis workflow.

11.1 Tool inventory

File	Type	Purpose
`fbo_pin.c` / `fbo_pin`	C program	The main tool. Five subcommands: `pin`, `app`, `observe`, `unpin`, `list`.
`fbo_enum.sh`	Shell	Optional. Standalone enumeration (same logic as `fbo_pin app`'s first half). Useful when you only have a shell.
`fbo_inspect.sh`	Shell	Analysis tool. Launches an app, reads `/proc/<pid>/maps`, reports byte breakdown across `/data/app/`, `/system/`, `/apex/`, `/data/data/`.

The C program is the deliverable. The shell scripts are diagnostic aids — useful while developing and validating, not required at runtime in production.

11.2 The five subcommands of `fbo_pin`

```
fbo_pin pin     <record-file> <file>...               pin explicit files
fbo_pin app     <record-file> <package>               static enum + pin
fbo_pin observe <record-file> <package> [<activity>]  launch + maps + pin
fbo_pin unpin   <record-file>                         reverse a previous pin
fbo_pin list    <record-file>                         dump the record
```

The two discovery modes — `app` (static, via `pm path` + directory walk) and `observe` (runtime, via `/proc/<pid>/maps`) — feed into the same pin backend. The relationship is covered in detail in Section 13.

11.3 `fbo_pin.c` (full source)

```
/*
 * fbo_pin.c — FBO 3.0 static-pin reference implementation.
 *
 * Build: gcc -O2 -Wall -o fbo_pin fbo_pin.c
 *
 * Per-file pin sequence:
 *   open(path, O_RDONLY)
 *     -> ioctl(fd, F2FS_IOC_SET_PIN_FILE, &one)    [best-effort]
 *     -> ioctl(fd, FS_IOC_FIEMAP, &fiemap)
 *     -> ufs_vendor_pin(extents)                   [stubbed]
 *     -> record (path, inode, mtime, extents)
 *
 * Commands:
 *   pin     <record-file> <file>...
```

```
 *   app     <record-file> <package>
 *   observe <record-file> <package> [<activity>]
 *   unpin   <record-file>
 *   list    <record-file>
 */

#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <dirent.h>
#include <libgen.h>
#include <sys/ioctl.h>
#include <sys/stat.h>
#include <linux/types.h>
#include <linux/fs.h>
#include <linux/fiemap.h>

#ifndef F2FS_IOC_SET_PIN_FILE
#define F2FS_IOC_SET_PIN_FILE _IOW(0xf5, 13, __u32)
#endif

#define MAX_EXTENTS 1024

/* ----- Generic helpers ----- */

static int popen_lines(const char *cmd, char ***out)
{
    FILE *p = popen(cmd, "r");
    if (!p) return -1;
    char **arr = NULL;
    int n = 0, cap = 0;
    char buf[4096];
    while (fgets(buf, sizeof(buf), p)) {
        size_t len = strlen(buf);
        while (len > 0 && (buf[len-1] == '\n' || buf[len-1] == '\r'))
            buf[--len] = 0;
        if (len == 0) continue;
        if (n == cap) {
            cap = cap ? cap * 2 : 16;
            arr = realloc(arr, cap * sizeof(*arr));
        }
        arr[n++] = strdup(buf);
    }
    pclose(p);
    *out = arr;
    return n;
}

static void free_lines(char **arr, int n) {
    if (!arr) return;
    for (int i = 0; i < n; i++) free(arr[i]);
    free(arr);
}

static void dir_append_files(const char *dir,
                             char ***vec, int *n, int *cap)
{
    DIR *d = opendir(dir);
    if (!d) return;
    struct dirent *e;
    while ((e = readdir(d))) {
        if (e->d_name[0] == '.') continue;
        char path[2048];
        snprintf(path, sizeof(path), "%s/%s", dir, e->d_name);
        struct stat st;
        if (stat(path, &st) < 0 || !S_ISREG(st.st_mode)) continue;
        if (*n == *cap) {
```

```
            *cap = *cap ? *cap * 2 : 16;
            *vec = realloc(*vec, *cap * sizeof(**vec));
        }
        (*vec)[(*n)++] = strdup(path);
    }
    closedir(d);
}

/* ----- Android-specific helpers ----- */

static const char *abi_to_isa(const char *abi)
{
    if (strcmp(abi, "arm64-v8a") == 0)   return "arm64";
    if (strcmp(abi, "armeabi-v7a") == 0) return "arm";
    return abi;
}

static int read_device_abi(char *out, size_t outlen)
{
    char **lines = NULL;
    int n = popen_lines("getprop ro.product.cpu.abi", &lines);
    if (n < 1) { free_lines(lines, n); return -1; }
    snprintf(out, outlen, "%s", lines[0]);
    free_lines(lines, n);
    return 0;
}

/* Enumerate the static distribution file set for an Android package:
 *   base.apk + every split + oat/<isa>/* + lib/<isa>/* (if extracted)
 */
static int enumerate_package(const char *pkg, char ***out)
{
    char cmd[256];
    snprintf(cmd, sizeof(cmd), "pm path %s 2>/dev/null", pkg);
    char **lines = NULL;
    int nl = popen_lines(cmd, &lines);
    if (nl < 1) {
        fprintf(stderr,
            "enumerate: package not installed or pm unavailable: %s\n",
            pkg);
        free_lines(lines, nl);
        return -1;
    }
    char **files = NULL;
    int nf = 0, cap = 0;
    for (int i = 0; i < nl; i++) {
        const char *p = lines[i];
        if (strncmp(p, "package:", 8) == 0) p += 8;
        if (nf == cap) {
            cap = cap ? cap * 2 : 16;
            files = realloc(files, cap * sizeof(*files));
        }
        files[nf++] = strdup(p);
    }
    char dir[1024];
    snprintf(dir, sizeof(dir), "%s", files[0]);
    char *slash = strrchr(dir, '/');
    if (slash) *slash = 0;
    free_lines(lines, nl);

    char abi[64];
    if (read_device_abi(abi, sizeof(abi)) == 0) {
        const char *isa = abi_to_isa(abi);
        char sub[2048];
        snprintf(sub, sizeof(sub), "%s/oat/%s", dir, isa);
        dir_append_files(sub, &files, &nf, &cap);
        snprintf(sub, sizeof(sub), "%s/lib/%s", dir, isa);
        dir_append_files(sub, &files, &nf, &cap);
    } else {
        fprintf(stderr,
            "enumerate: ro.product.cpu.abi unavailable; "
```

```
                "skipping oat/ and lib/\n");
        }
        *out = files;
        return nf;
    }

    /* Read /proc/<pid>/maps and return a deduplicated list of mapped
     * file paths matching the given prefix. */
    static int read_proc_maps(int pid, const char *prefix, char ***out)
    {
        char path[64];
        snprintf(path, sizeof(path), "/proc/%d/maps", pid);
        FILE *f = fopen(path, "r");
        if (!f) {
            fprintf(stderr, "open %s: %s\n", path, strerror(errno));
            return -1;
        }
        char **vec = NULL;
        int n = 0, cap = 0;
        char line[4096];
        while (fgets(line, sizeof(line), f)) {
            size_t len = strlen(line);
            while (len > 0 && (line[len-1] == '\n' || line[len-1] == '\r'))
                line[--len] = 0;
            char *p = strrchr(line, ' ');
            if (!p) continue;
            p++;
            if (*p != '/') continue;
            if (strncmp(p, "/dev/", 5) == 0) continue;
            if (strncmp(p, "/memfd:", 7) == 0) continue;
            if (strncmp(p, prefix, strlen(prefix)) != 0) continue;
            int dup = 0;
            for (int i = 0; i < n; i++)
                if (strcmp(vec[i], p) == 0) { dup = 1; break; }
            if (dup) continue;
            if (n == cap) {
                cap = cap ? cap * 2 : 16;
                vec = realloc(vec, cap * sizeof(*vec));
            }
            vec[n++] = strdup(p);
        }
        fclose(f);
        *out = vec;
        return n;
    }

    static int resolve_activity(const char *pkg, char *out, size_t outlen)
    {
        char cmd[256];
        snprintf(cmd, sizeof(cmd),
            "cmd package resolve-activity --brief %s 2>/dev/null", pkg);
        char **lines = NULL;
        int n = popen_lines(cmd, &lines);
        if (n < 1) { free_lines(lines, n); return -1; }
        const char *comp = lines[n - 1];
        if (!strchr(comp, '/')) { free_lines(lines, n); return -1; }
        snprintf(out, outlen, "%s", comp);
        free_lines(lines, n);
        return 0;
    }

    static int wait_for_pid(const char *pkg, int secs)
    {
        char cmd[128];
        snprintf(cmd, sizeof(cmd), "pidof %s 2>/dev/null", pkg);
        for (int i = 0; i < secs; i++) {
            char **lines = NULL;
            int n = popen_lines(cmd, &lines);
            if (n >= 1) {
                int pid = atoi(lines[0]);
                free_lines(lines, n);
```

```
            if (pid > 0) return pid;
        }
        free_lines(lines, n);
        sleep(1);
    }
    return -1;
}

/* ----- Pin / FIEMAP / record ----- */

static int do_f2fs_pin(int fd, const char *path, int pin)
{
    __u32 v = pin ? 1 : 0;
    if (ioctl(fd, F2FS_IOC_SET_PIN_FILE, &v) < 0) {
        if (errno != ENOTTY)
            fprintf(stderr, "  warning: F2FS pin on %s: %s\n",
                    path, strerror(errno));
        return -1;
    }
    return 0;
}

static int fiemap_file(int fd, const char *path,
                       struct fiemap_extent *out, unsigned *out_n)
{
    size_t bytes = sizeof(struct fiemap) +
                   MAX_EXTENTS * sizeof(struct fiemap_extent);
    struct fiemap *fm = calloc(1, bytes);
    if (!fm) { perror("calloc"); return -1; }
    fm->fm_start        = 0;
    fm->fm_length       = ~0ULL;
    fm->fm_flags        = FIEMAP_FLAG_SYNC;
    fm->fm_extent_count = MAX_EXTENTS;
    if (ioctl(fd, FS_IOC_FIEMAP, fm) < 0) {
        fprintf(stderr, "  FIEMAP on %s: %s\n", path, strerror(errno));
        free(fm);
        return -1;
    }
    unsigned n = fm->fm_mapped_extents;
    if (n > MAX_EXTENTS) n = MAX_EXTENTS;
    memcpy(out, fm->fm_extents, n * sizeof(struct fiemap_extent));
    *out_n = n;
    free(fm);
    return 0;
}

static int ufs_vendor_pin(const char *path,
                          const struct fiemap_extent *exts, unsigned n,
                          int pin)
{
    const char *op = pin ? "PIN" : "UNPIN";
    (void)path;
    for (unsigned i = 0; i < n; i++) {
        printf("  ufs_%s  lba=0x%llx  len=0x%llx  flags=0x%x\n",
               op,
               (unsigned long long)exts[i].fe_physical,
               (unsigned long long)exts[i].fe_length,
               exts[i].fe_flags);
    }
    return 0;
}

static int record_write(FILE *rec, const char *path,
                        ino_t ino, time_t mtime,
                        const struct fiemap_extent *exts, unsigned n)
{
    fprintf(rec, "FILE %s ino=%llu mtime=%lld nextents=%u\n",
            path, (unsigned long long)ino,
            (long long)mtime, n);
    for (unsigned i = 0; i < n; i++) {
        fprintf(rec, "  EXT phys=0x%llx len=0x%llx flags=0x%x\n",
```

```
                (unsigned long long)exts[i].fe_physical,
                (unsigned long long)exts[i].fe_length,
                exts[i].fe_flags);
    }
    return 0;
}

static int pin_files(const char *recpath, char **paths, int npaths)
{
    FILE *rec = fopen(recpath, "w");
    if (!rec) { perror(recpath); return 1; }
    int ok = 0;
    for (int i = 0; i < npaths; i++) {
        const char *path = paths[i];
        printf("PIN %s\n", path);
        int fd = open(path, O_RDONLY);
        if (fd < 0) {
            fprintf(stderr, "  open: %s\n", strerror(errno));
            continue;
        }
        struct stat st;
        if (fstat(fd, &st) < 0) {
            perror("  fstat"); close(fd); continue;
        }
        do_f2fs_pin(fd, path, 1);
        struct fiemap_extent exts[MAX_EXTENTS];
        unsigned n = 0;
        if (fiemap_file(fd, path, exts, &n) < 0) {
            close(fd); continue;
        }
        ufs_vendor_pin(path, exts, n, 1);
        record_write(rec, path, st.st_ino, st.st_mtime, exts, n);
        close(fd);
        printf("  done -- %u extent(s) pinned\n\n", n);
        ok++;
    }
    fclose(rec);
    printf("pinned %d / %d files; record at %s\n", ok, npaths, recpath);
    return 0;
}

/* ----- Commands ----- */

static int cmd_pin(const char *recpath, char **paths, int npaths)
{
    return pin_files(recpath, paths, npaths);
}

static int cmd_app(const char *recpath, const char *pkg)
{
    char **files = NULL;
    int n = enumerate_package(pkg, &files);
    if (n < 0) return 1;
    printf("enumerated %d file(s) for %s:\n", n, pkg);
    for (int i = 0; i < n; i++) printf("  %s\n", files[i]);
    printf("\n");
    int rc = pin_files(recpath, files, n);
    for (int i = 0; i < n; i++) free(files[i]);
    free(files);
    return rc;
}

static int cmd_observe(const char *recpath, const char *pkg,
                       const char *activity_opt)
{
    char activity[256];
    if (activity_opt && strchr(activity_opt, '/')) {
        snprintf(activity, sizeof(activity), "%s", activity_opt);
    } else if (resolve_activity(pkg, activity, sizeof(activity)) < 0) {
        fprintf(stderr,
            "observe: could not resolve activity for %s; "
```

```
                "pass it as 4th arg\n", pkg);
            return 1;
        }
        char cmd[512];
        int unused;
        snprintf(cmd, sizeof(cmd), "am force-stop %s >/dev/null 2>&1", pkg);
        unused = system(cmd);
        sleep(1);
        snprintf(cmd, sizeof(cmd), "am start -n %s >/dev/null 2>&1", activity);
        unused = system(cmd);
        (void)unused;
        int pid = wait_for_pid(pkg, 10);
        if (pid < 0) {
            fprintf(stderr, "observe: process never appeared for %s\n", pkg);
            return 1;
        }
        sleep(3);
        char **files = NULL;
        int n = read_proc_maps(pid, "/data/app/", &files);
        if (n < 0) return 1;
        if (n == 0) {
            fprintf(stderr,
                "observe: no /data/app/ files mapped\n");
            return 1;
        }
        printf("observed %d mapped file(s) under /data/app/ for %s (pid %d):\n",
               n, pkg, pid);
        for (int i = 0; i < n; i++) printf("  %s\n", files[i]);
        printf("\n");
        int rc = pin_files(recpath, files, n);
        for (int i = 0; i < n; i++) free(files[i]);
        free(files);
        return rc;
    }

    static int cmd_unpin(const char *recpath)
    {
        FILE *rec = fopen(recpath, "r");
        if (!rec) { perror(recpath); return 1; }
        char line[1024], path[512] = {0};
        while (fgets(line, sizeof(line), rec)) {
            if (strncmp(line, "FILE ", 5) == 0) {
                sscanf(line, "FILE %511s", path);
                printf("UNPIN %s\n", path);
            } else if (strncmp(line, "  EXT ", 6) == 0) {
                struct fiemap_extent e = {0};
                unsigned long long phys, len; unsigned flags;
                if (sscanf(line, "  EXT phys=0x%llx len=0x%llx flags=0x%x",
                           &phys, &len, &flags) == 3) {
                    e.fe_physical = phys;
                    e.fe_length   = len;
                    e.fe_flags    = flags;
                    ufs_vendor_pin(path, &e, 1, 0);
                }
            }
        }
        rewind(rec);
        while (fgets(line, sizeof(line), rec)) {
            if (strncmp(line, "FILE ", 5) != 0) continue;
            sscanf(line, "FILE %511s", path);
            int fd = open(path, O_RDONLY);
            if (fd >= 0) { do_f2fs_pin(fd, path, 0); close(fd); }
        }
        fclose(rec);
        return 0;
    }

    static int cmd_list(const char *recpath)
    {
        FILE *rec = fopen(recpath, "r");
        if (!rec) { perror(recpath); return 1; }
```

```
    char line[1024];
    while (fgets(line, sizeof(line), rec)) fputs(line, stdout);
    fclose(rec);
    return 0;
}

int main(int argc, char **argv)
{
    if (argc < 3) {
        fprintf(stderr,
            "usage:\n"
            "  %s pin     <record-file> <file>...\n"
            "  %s app     <record-file> <package>\n"
            "  %s observe <record-file> <package> [<activity>]\n"
            "  %s unpin   <record-file>\n"
            "  %s list    <record-file>\n",
            argv[0], argv[0], argv[0], argv[0], argv[0]);
        return 1;
    }
    if (strcmp(argv[1], "pin") == 0 && argc >= 4)
        return cmd_pin(argv[2], &argv[3], argc - 3);
    if (strcmp(argv[1], "app") == 0 && argc == 4)
        return cmd_app(argv[2], argv[3]);
    if (strcmp(argv[1], "observe") == 0 && (argc == 4 || argc == 5))
        return cmd_observe(argv[2], argv[3], argc == 5 ? argv[4] : NULL);
    if (strcmp(argv[1], "unpin") == 0) return cmd_unpin(argv[2]);
    if (strcmp(argv[1], "list") == 0)  return cmd_list(argv[2]);
    fprintf(stderr, "unknown command or wrong arity: %s\n", argv[1]);
    return 1;
}
```

11.4 `fbo_enum.sh` (shell-only enumeration)

```
#!/system/bin/sh
# fbo_enum.sh — enumerate the static distribution-side files of a
# package. Output: one absolute path per line. Pipe to fbo_pin pin.

PKG=$1
[ -z "$PKG" ] && { echo "usage: $0 <package>" >&2; exit 1; }

ABI=$(getprop ro.product.cpu.abi)
case "$ABI" in
    arm64-v8a)   ISA=arm64 ;;
    armeabi-v7a) ISA=arm   ;;
    x86_64)      ISA=x86_64 ;;
    x86)         ISA=x86   ;;
    *)           ISA=$ABI  ;;
esac

APKS=$(pm path "$PKG" 2>/dev/null | sed 's/^package://')
[ -z "$APKS" ] && { echo "package not installed: $PKG" >&2; exit 1; }

DIR=$(dirname "$(echo "$APKS" | head -n1)")

echo "$APKS"
ls "$DIR/oat/$ISA"/* 2>/dev/null
ls "$DIR/lib/$ISA"/* 2>/dev/null
```

11.5 `fbo_inspect.sh` (runtime analysis)

This script launches an app, snapshots its memory map, and reports the byte breakdown across the directories that matter. It is the tool that generates the app-vs-framework byte ratio discussed in Section 12.

```
#!/system/bin/sh
# fbo_inspect.sh — launch an Android app and summarize what files it
# has mmap'd, broken down by where on disk they live.
```

```
PKG=$1
ACT=$2

if [ -z "$PKG" ]; then
    echo "usage: $0 <package> [<activity-component>]" >&2
    exit 1
fi

file_size() {
    s=$(stat -c '%s' "$1" 2>/dev/null)
    if [ -z "$s" ]; then s=$(wc -c < "$1" 2>/dev/null); fi
    echo "${s:-0}"
}

sum_sizes() {
    total=0
    while read -r f; do
        [ -f "$f" ] || continue
        s=$(file_size "$f")
        total=$((total + s))
    done
    echo "$total"
}

mb() { awk -v b="$1" 'BEGIN { printf "%.1f MB", b/1048576 }'; }

if [ -z "$ACT" ]; then
    ACT=$(cmd package resolve-activity --brief "$PKG" 2>/dev/null \
          | tail -1 | tr -d '\r')
fi
if [ -z "$ACT" ] || ! echo "$ACT" | grep -q '/'; then
    echo "could not resolve activity for $PKG" >&2
    exit 1
fi

am force-stop "$PKG"; sleep 1
am start -n "$ACT" >/dev/null 2>&1

PID=""
for i in 1 2 3 4 5 6 7 8 9 10; do
    PID=$(pidof "$PKG" 2>/dev/null)
    [ -n "$PID" ] && break
    sleep 1
done
[ -z "$PID" ] && { echo "process never appeared" >&2; exit 1; }

sleep 3

MAPS=/data/local/tmp/fbo_inspect_$PID.maps
cat /proc/$PID/maps | awk '{print $NF}' | sort -u \
    | grep -v '^\[' | grep -v '^$' \
    | grep -v '^/memfd' | grep -v '^/dev/' > "$MAPS"

report_category() {
    label=$1
    pattern=$2
    files=$(grep -E "$pattern" "$MAPS")
    count=$(echo "$files" | grep -c .)
    bytes=$(echo "$files" | sum_sizes)
    echo "[$label]"
    echo "  files : $count"
    echo "  bytes : $bytes ($(mb $bytes))"
}

report_category "/data/app/ (per-app, FBO target)"  '^/data/app/'
report_category "/system/ + /apex/ (framework)"     '^/system/|^/apex/'
report_category "/data/data/ (runtime data)"        '^/data/data/'
report_category "/data/dalvik-cache/ (boot OAT)"    '^/data/dalvik-cache/'

echo
echo "[/data/app/ files in detail]"
```

```
        grep '^/data/app/' "$MAPS" | while read -r f; do
            [ -f "$f" ] || continue
            s=$(file_size "$f")
            printf "  %10d  %s\n" "$s" "$f"
        done | sort -rn

        rm -f "$MAPS"
```

11.6 Building and deploying

On a Linux workstation (x86 testing or NDK cross-build):

```
# Native x86 build (for testing the pin/list/unpin path)
gcc -O2 -Wall -o fbo_pin fbo_pin.c

# Android cross-build with the NDK
$NDK/toolchains/llvm/prebuilt/linux-x86_64/bin/aarch64-linux-android30-clang \
    -O2 -Wall -o fbo_pin fbo_pin.c

# Push everything to a rooted dev device
adb push fbo_pin fbo_enum.sh fbo_inspect.sh /data/local/tmp/
adb shell chmod +x /data/local/tmp/fbo_pin /data/local/tmp/*.sh
```

Then either via adb shell or directly on the device console, as root:

```
cd /data/local/tmp

# Production-style: static enum + pin
./fbo_pin app  /data/local/tmp/antutu.rec  com.antutu.ABenchMark

# Validation-style: launch + observe + pin
./fbo_pin observe  /data/local/tmp/antutu_obs.rec  com.antutu.ABenchMark

# Analysis: byte breakdown across categories
./fbo_inspect.sh  com.antutu.ABenchMark

# Tear down
./fbo_pin unpin  /data/local/tmp/antutu.rec
```

11.7 What to change for production

The reference implementation is intentionally minimal. To go from this to a daemon shipped to OEMs:

1. **`ufs_vendor_pin()`** — replace the `printf` with the actual call to the OEM's pin interface (vendor sysfs node, scsi passthrough, or vendor ioctl). The function signature does not change.
2. **Record store** — replace the text record file with SQLite or a small binary format, keyed by package name.
3. **Lifecycle integration** — replace the CLI driver with a service subscribing to `ACTION_PACKAGE_REPLACED`, `ACTION_PACKAGE_REMOVED`, and OAT-regeneration signals.
4. **Pin budget management** — LRU eviction keyed on per-app launch timestamps from `usagestats`.
5. **SELinux policy** — vendor domain for the daemon.
6. **init.rc service definition** — standard plumbing.
7. **Telemetry** — per-app pin success/failure counts, region utilization, time-since-last-pin per app.

The core — the four-step per-file sequence in `pin_files()`, the two discovery functions, and the shared command dispatch — does not change.

11.8 A note on C versus C++

The implementation is C. C++ would offer `std::string`, `std::vector<std::string>`, `std::filesystem::directory_iterator`, RAII for `FILE *` and `int fd`. The wins are modest for a 600-line tool; the cost is a `libstdc++` / `libc++` dependency in the shipped binary. The ioctl-heavy parts (FIEMAP, F2FS pin, vendor UFS interface) are no cleaner in C++ than in C. Staying in C keeps dependencies

minimal and matches the style of the surrounding kernel-adjacent tooling. If a future version of the tool grows complex in-memory state — pin index with secondary keys for eviction policy, dependency graphs between pins — that's the point at which C++ rewrite would start paying for itself.

---

12. Cold-launch I/O — what actually hits disk

This section captures the observability work done during validation and the conclusions it produced about FBO 3.0's scope. The findings are non-obvious and worth recording because they explain why the static-pin architecture is correct even though it appears, at first glance, to address only a small fraction of an app's file mappings.

12.1 Why `strace` on `am start` is misleading

A natural first attempt at "what files does Antutu open?" is:

```
strace -f -e openat am start -n <pkg>/<activity>
```

This produces a thousand-line trace dominated by paths under `/dev/__properties__/`, `/system/lib64/`, `/odm/lib64/`, `/vendor/lib64/`, `/apex/com.android.*/lib64/`. None of those are Antutu's files.

The reason: `am` is a small CLI binary that sends a Binder message to `system_server`. `system_server` then asks `zygote64` to fork a new process, which becomes the app. The fork happens **after `am` has already exited**. So the strace captures only `am`'s own startup — the dynamic linker resolving `am`'s shared library dependencies, Bionic property service touches — and never sees the app process at all.

This is also why `wrap.<package>` with strace tends to fail in practice: it changes the launch path from a fork to an exec, breaks Zygote's expectations about the child's process state, and the app ends up crashing or being killed as ANR-adjacent before its real loads happen.

12.2 The Zygote architecture

The relevant Android internals:

At boot, `zygote64` (and on dual-ABI devices, `zygote` for 32-bit) starts. It does several things, then sits idle:

1. Maps the boot image: `/system/framework/<isa>/boot.art`, `boot.oat`, `boot.vdex`, plus a long list of related `boot-*` files.
2. Loads the framework JARs and their corresponding OAT artifacts — `framework.jar` and `framework.oat`, plus dozens of others.
3. `dlopen`s every commonly-used native library — `libart.so`, `libbinder.so`, `libgui.so`, `libsqlite.so`, codec libs, etc.
4. Pre-resolves a curated set of classes and methods (the "preload classes" list).
5. Waits on a socket for app-launch requests from `system_server`.

When an app is launched, Zygote does NOT exec a new binary. It does `fork()`, the child specializes itself (sets SELinux context, sandboxes, calls into the activity main), and runs as the app. All of Zygote's prior memory mappings — the entire framework, every preloaded `.so`, the boot image — are inherited by the child via standard copy-on-write semantics.

For read-only sections of those files (which is essentially all of them), the inherited mappings reference the *same physical pages in RAM* as Zygote. No I/O is done at app launch to read those files — they are already resident, and the new process has a virtual mapping to the same physical memory.

This means:

- A file appearing in `/proc/$PID/maps` does **not** imply that file was read from disk during this app's launch.
- The framework's bytes are paid once, at boot, by Zygote.
- Every app launch after that inherits the warmth for free.

12.3 `/proc/<pid>/maps` as the ground-truth tool

For "what files is this app using right now," `/proc/<pid>/maps` is both more accurate and easier than tracing. Every file the app uses for code or resources is mmap'd (APKs, OAT, .so, all of it), and mmap'd files persist in the map until close. So a snapshot taken after launch settles gives the complete file set.

```
PID=$(pidof com.antutu.ABenchMark)
cat /proc/$PID/maps | awk '{print $NF}' | sort -u | grep '/data/app/'
```

This is what `fbo_pin observe` does in C, and what `fbo_inspect.sh` formats with size and category breakdowns.

The C application uses this in `read_proc_maps()` (Section 11.3).

12.4 Interpreting the app vs framework byte ratio

A typical measurement on a modern Android device, for a benchmark or moderate-sized app:

```
/data/app/         ~ 100-300 MB     (per-app, cold on first launch)
/system/ + /apex/  ~ 200-400 MB     (framework, already warm via Zygote)
/data/data/        ~  10-50  MB     (runtime data, mostly small writes)
/data/dalvik-cache ~ 100-200 MB     (boot OAT, already warm via Zygote)
```

It is tempting to compute "app share = `/data/app/` / total" and worry that the number is small. That number is **misleading**. The correct denominator for "what fraction of cold-launch I/O does FBO address?" is not total mapped bytes but cold mapped bytes.

By construction:

- `/data/app/` is cold the first time the app launches after page cache eviction. Every byte is real disk I/O.
- `/system/` and `/apex/` are pre-mapped by Zygote at boot. Cold I/O cost during app launch is approximately zero.
- `/data/dalvik-cache/` is pre-mapped by Zygote for the boot image set. Same as above.
- `/data/data/` involves writes more than reads, and the reads tend to be small and lazy. Marginal contribution to cold launch.

So the "app share of cold-launch I/O" is closer to **near-100%** of what FBO 3.0 actually has the leverage to optimize. The remaining ~0% is platform-level pinning (boot image, framework) which is the OEM's responsibility at platform integration time and is typically already addressed by the time FBO 3.0 ships.

12.5 When the assumption breaks

The "framework is always in RAM via Zygote" assumption is robust but not absolute. Cases where it fails:

1. **First app launch after cold boot.** Zygote pays the framework load cost once. The very first app launch post-boot inherits a freshly-loaded Zygote, but the kernel hasn't fully filled the working set yet. The next several launches pay a small amortized cost as Zygote's preload set finishes faulting in.
2. **Lazily-loaded platform libraries.** Codec libs for unusual codecs, ML accelerator drivers, certain `dex2oat` plugins — these are not in Zygote's preload set and get loaded on first use. The cost is amortized over the device's lifetime, not per-app.
3. **Severe memory pressure.** On low-RAM devices under heavy multitasking, the kernel can evict clean file-backed pages despite high reference counts. Zygote's mappings are partially protected (high refcount, frequent reuse), but not guaranteed.
4. **Background `dex2oat` regeneration.** After OTAs or install updates, the system may rewrite `.odex` files in the background. The new file's pages are not yet cached; first launch after regeneration is colder than usual.

None of these change the FBO 3.0 design. They are real costs that exist alongside FBO and are addressed (or accepted) elsewhere in the platform.

12.6 What this means for FBO 3.0 scope

The architecture that falls out of this analysis:

- FBO 3.0 pins **per-app distribution files** under `/data/app/`.
- It does **not** pin framework files. Those are platform-level.
- It does **not** pin runtime data files. Those are out of scope.
- The per-app pin region budget can be small (low hundreds of MB per app), because the set of files per app is small.
- The pin region budget can be amortized across many apps with LRU eviction, because most apps launch on a long-tail distribution while a small set of "hot" apps gets ~all the launches.

This is exactly the per-app distribution-file pin that the `fbo_pin app` subcommand implements.

---

13. Two discovery paths

The C application exposes two ways to determine which files to pin for a given package. They are complementary and the tool ships with both.

13.1 Static enumeration: `fbo_pin app`

Implementation: `enumerate_package()` in `fbo_pin.c`.

Steps:

1. Call `pm path <package>` via `popen()`. Strip `package:` prefix from each output line. This yields `base.apk` and every installed split.
2. Derive the install directory by taking `dirname()` of the first APK path.
3. Read `ro.product.cpu.abi` via `getprop`, map to ISA short form (`arm64-v8a` -> `arm64`, etc.).
4. List `<install-dir>/oat/<isa>/*` — these are the AOT compiled artifacts (`.odex`, `.vdex`, `.art`).
5. List `<install-dir>/lib/<isa>/*` — extracted native libraries, if the app has `extractNativeLibs="true"` in its manifest. For modern apps with `extractNativeLibs="false"`, this directory does not exist and the libs are mmap'd directly from inside the APK.

Result: typically 6-15 absolute paths.

Properties: fast (milliseconds), no app launch required, no side effects, deterministic given a particular install.

When to use: production. This is what an OEM service calls on `ACTION_PACKAGE_REPLACED` or at hot-list boot setup.

13.2 Runtime observation: `fbo_pin observe`

Implementation: `cmd_observe()` in `fbo_pin.c`.

Steps:

1. Resolve the launchable activity via `cmd package resolve-activity --brief`, or accept it as an argument.
2. `am force-stop` the package, sleep briefly.
3. `am start` the resolved activity.
4. Poll `pidof <package>` for up to 10 seconds to wait for the process to appear.
5. Sleep 3 seconds to let launch-phase mmaps complete.
6. Open `/proc/<pid>/maps`, parse the last whitespace-separated field of each line, dedupe, filter to paths under `/data/app/`.
7. Pin each file using the same `pin_files()` backend.

Result: typically the same 6-15 paths, plus any additional `/data/app/`-rooted files the app loads at runtime that static enumeration missed.

Properties: requires an app launch (the launch itself is cold; the pin benefits subsequent launches). Captures ground truth — anything the app actually maps appears in `/proc/<pid>/maps`.

When to use: validation, refinement, debugging. Run it once per app or per Android version to confirm the static path is complete; if it isn't, that's a static-path bug to fix.

13.3 When to use which

Scenario	Path
Production pin on package install	`app`
Production pin on system boot	`app`
Validating the static path is complete	`observe`, then `diff` against `app`
Investigating a new Android version's file layout	`observe`
Investigating an app that loads files via unusual paths	`observe`
One-shot pin from a CLI on a dev device	either

The `diff` workflow:

```
./fbo_pin app     /tmp/static.rec  com.antutu.ABenchMark
./fbo_pin unpin   /tmp/static.rec
./fbo_pin observe /tmp/observed.rec com.antutu.ABenchMark

./fbo_pin list /tmp/static.rec   | grep '^FILE ' | awk '{print $2}' | sort > /tmp/s.txt
./fbo_pin list /tmp/observed.rec | grep '^FILE ' | awk '{print $2}' | sort > /tmp/o.txt
diff /tmp/s.txt /tmp/o.txt
```

Empty diff means static enumeration is correct and complete for this app. Any difference is actionable — extend `enumerate_package()` or treat the app as an exception.

13.4 Shared pin backend

Both modes call into `pin_files()`. Per file, the sequence is:

```
open(path, O_RDONLY)
fstat(fd, &st)                                  // for record metadata
ioctl(fd, F2FS_IOC_SET_PIN_FILE, &one)          // prevent F2FS GC
ioctl(fd, FS_IOC_FIEMAP, &fm)                   // get LBA extents
ufs_vendor_pin(extents)                         // STUB; OEM-specific
record_write(rec, path, ino, mtime, extents)    // for later unpin
close(fd)
```

The only OEM-specific code is `ufs_vendor_pin()`. Everything else is standard Linux. Wiring the stub to the real pin interface is the only work required to make either discovery mode produce real pins on a real device.

---

14. Required files and tools

This section is an inventory: the complete set of files needed to build, deploy, and use FBO 3.0 as documented here.

14.1 Source files

File	Purpose	Lines	Required?
`fbo_pin.c`	The C tool implementing all five subcommands.	~600	Yes

`fbo_enum.sh`	Shell-only static enumeration. Same logic as `fbo_pin app`.	~25	Optional
`fbo_inspect.sh`	Runtime byte-breakdown analysis tool.	~80	Recommended for validation

14.2 Build artifacts

Artifact	How built	Where it runs
`fbo_pin` (x86 binary)	`gcc -O2 -Wall -o fbo_pin fbo_pin.c`	Linux workstation, for local testing of pin/list/unpin
`fbo_pin` (arm64 Android binary)	NDK clang cross-compile	Android device

The same `fbo_pin.c` source compiles to both. There are no architecture-specific code paths; all the Android-specific behavior is factored through helpers (`popen("pm path ...")`, `popen("getprop ...")`, `read_proc_maps`) which return clean errors on non-Android hosts.

14.3 Runtime prerequisites on Android

Item	Where it comes from	Why
Root access	adb root, or device console as root	Required for all ioctl operations and `/proc/<pid>/maps` reading across SELinux domains
`pm` binary	Android platform (always present)	Used by `fbo_pin app` for static enumeration
`am` binary	Android platform (always present)	Used by `fbo_pin observe` to launch apps
`getprop` binary	Android platform (always present)	Used to read `ro.product.cpu.abi`
`cmd` binary	Android platform (Android 7+)	Used by `fbo_pin observe` for `cmd package resolve-activity`
F2FS filesystem at `/data`	Android default since ~Android 8	For `F2FS_IOC_SET_PIN_FILE` to take effect; on ext4 the pin is silently no-op'd
`CONFIG_F2FS_FS_PIN` in kernel	OEM kernel build	Enables the pin ioctl in F2FS
OEM UFS pin interface	OEM-specific	The actual binding for `ufs_vendor_pin()` — sysfs node, scsi passthrough, or vendor ioctl

14.4 Validation workflow

To validate FBO 3.0 end-to-end on a dev device:

```
# 1. Build and deploy
gcc -O2 -Wall -o fbo_pin fbo_pin.c            # or NDK cross-compile
adb push fbo_pin fbo_inspect.sh /data/local/tmp/
adb shell chmod +x /data/local/tmp/fbo_pin /data/local/tmp/fbo_inspect.sh

# 2. Inspect the target app's footprint
adb shell /data/local/tmp/fbo_inspect.sh com.antutu.ABenchMark
# Read: app share of mapped bytes, file list, top largest files

# 3. Compare static vs observed enumeration
adb shell /data/local/tmp/fbo_pin app     /data/local/tmp/s.rec com.antutu.ABenchMark
adb shell /data/local/tmp/fbo_pin unpin   /data/local/tmp/s.rec
adb shell /data/local/tmp/fbo_pin observe /data/local/tmp/o.rec com.antutu.ABenchMark
```

```
# 4. Pin and A/B measure cold launch time
adb shell /data/local/tmp/fbo_pin unpin /data/local/tmp/o.rec

# Unpinned baseline
for i in 1 2 3 4 5; do
    adb shell am force-stop com.antutu.ABenchMark
    adb shell 'echo 3 > /proc/sys/vm/drop_caches'
    adb shell am start -W -n
com.antutu.ABenchMark/com.android.module.app.ui.start.ABenchMarkStart \
        | grep -E 'TotalTime|WaitTime'
    sleep 3
done

# Pinned
adb shell /data/local/tmp/fbo_pin app /data/local/tmp/p.rec com.antutu.ABenchMark
for i in 1 2 3 4 5; do
    adb shell am force-stop com.antutu.ABenchMark
    adb shell 'echo 3 > /proc/sys/vm/drop_caches'
    adb shell am start -W -n
com.antutu.ABenchMark/com.android.module.app.ui.start.ABenchMarkStart \
        | grep -E 'TotalTime|WaitTime'
    sleep 3
done

# Tear down
adb shell /data/local/tmp/fbo_pin unpin /data/local/tmp/p.rec
```

The delta in median `TotalTime` between the two batches is the end-to-end evidence for FBO 3.0's impact.

14.5 Summary of the work

- **`fbo_pin.c`** is the C application. Five subcommands, one shared pin backend, two discovery modes (static and runtime). Ships as a single binary, no library dependencies, ~600 lines.
- **`fbo_enum.sh`** is the equivalent shell-only enumerator. Helpful on devices without the binary, optional otherwise.
- **`fbo_inspect.sh`** is the analysis tool that produced the app-vs-framework byte-ratio measurements used to validate FBO 3.0's scope.
- The architectural conclusion — pin `/data/app/` per-app, leave `/system/` to the platform — is grounded in the Zygote sharing model documented in Section 12.
- The reference implementation needs exactly one substitution to be production-ready: wiring `ufs_vendor_pin()` to the OEM pin interface.

---

15. Kernel-side implementation

The userspace tool documented in Section 11 is the right shape for development, debugging, and rapid iteration. For production deployment on OEM devices, there is a complementary path: a kernel module that exposes a sysfs entry and performs the entire enumeration + pin sequence in kernel context. This section documents that module — its architecture, its source, how to build it, and the open wiring tasks that remain.

15.1 Why an in-kernel path is worth having

The userspace tool is fragile in ways specific to Android production:

- A long-running daemon needs `init.rc`, SELinux policy, restart semantics, and survives memory pressure only by careful design.
- Each pin operation crosses the userspace/kernel boundary several times (`pm path` exec, `open`, `ioctl` for F2FS pin, `ioctl` for FIEMAP, write to sysfs/scsi for UFS pin). Microseconds individually, but the count adds up under load.
- Userspace tracing infrastructure (`strace`, `atrace`, `perfetto`) is unevenly available across OEM kernels, making field debugging difficult.

An in-kernel module addresses all three. It loads once at boot, has no SELinux policy concerns for filesystem access (the kernel owns the namespace), and avoids the cross-boundary round trips on the hot path. The interface to userspace collapses to a single sysfs write:

```
echo com.antutu.ABenchMark > /sys/kernel/fbo/pin
```

15.2 Why this does NOT go in F2FS itself

A natural-sounding place for this code is `fs/f2fs/`. That would be wrong, for reasons worth recording:

1. **F2FS is a generic Linux filesystem**, used outside Android in embedded distros and Chrome OS. Embedding Android-specific path conventions (`/data/app/<H1>/<pkg>-<H2>/`) inside F2FS forks the filesystem from upstream and couples it to an application framework.
2. **"Package" is a userspace concept**, owned by `PackageManagerService` in `system_server`. F2FS has no business knowing about it.
3. **Path layout changes between Android versions**. Putting the knowledge in F2FS means every Android directory-layout change is an F2FS patch. Putting it in a separate vendor module means the F2FS source stays clean and the vendor module rebases freely.
4. **Upstream F2FS maintainers will reject Android-aware patches**. The Kim et al at Samsung who maintain upstream F2FS actively redirect this kind of code to vendor modules.

The correct division of responsibility:

```
userspace              decides WHAT to pin   (which package, which policy)
fbo_module.ko          does the enumeration  (walk /data/app/, find files)
F2FS                   provides mechanism    (existing pin ioctl,
                                               segment allocation hints)
hardware (UFS)         provides residency    (vendor pin region)
```

`fbo_module.ko` is a separate kernel module — not a patch to `fs/f2fs/`. F2FS continues to expose its existing per-file pin ioctl. The module calls into F2FS through that same interface a userspace caller would use.

15.3 The four kernel API building blocks

The module needs to do four things in kernel context. Each maps to a well-known kernel API.

Operation	API	Notes
Walk a directory one level	`iterate_dir()` with a `struct dir_context` callback	`filldir_t` return type changed `int` → `bool` around kernel 6.0; target 5.10+
Open a file by absolute path	`filp_open()`	Returns IS_ERR pointer; check with `IS_ERR(filp)` ; close with `filp_close()`
Extract physical extent map	`inode->i_op->fiemap()` or filesystem-specific helper like F2FS's `f2fs_map_blocks()`	Generic FIEMAP path declares its output buffer `__user` ; in kernel context this requires either a small patch to skip the access check or use of the filesystem-specific helper
F2FS pin	None exported today	Either patch F2FS to `EXPORT_SYMBOL_GPL` a helper, or skip F2FS pin and accept GC drift

The first three are unproblematic. The fourth is the one piece of F2FS-specific work that the module needs from the filesystem.

15.4 Module source

```
        // SPDX-License-Identifier: GPL-2.0
```

```c
/*
 * fbo_module.c — FBO 3.0 in-kernel pin module.
 *
 * Exposes /sys/kernel/fbo/pin. Writing a package name to it triggers
 * the kernel-side enumeration and pin sequence for that package's
 * static distribution files under /data/app/.
 *
 *   echo com.antutu.ABenchMark > /sys/kernel/fbo/pin
 *
 * Targets kernel >= 5.10.
 */

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/kobject.h>
#include <linux/sysfs.h>
#include <linux/slab.h>
#include <linux/string.h>
#include <linux/fs.h>
#include <linux/file.h>
#include <linux/namei.h>
#include <linux/fiemap.h>
#include <linux/cred.h>

#define FBO_DATA_APP   "/data/app"
#define FBO_MAX_PATH   1024
#define FBO_MAX_NAME   256
#define FBO_MAX_FILES  64
#define FBO_MAX_EXT    256

#ifdef CONFIG_ARM64
  #define FBO_ISA "arm64"
#elif defined(CONFIG_ARM)
  #define FBO_ISA "arm"
#elif defined(CONFIG_X86_64)
  #define FBO_ISA "x86_64"
#elif defined(CONFIG_X86)
  #define FBO_ISA "x86"
#else
  #define FBO_ISA "unknown"
#endif

static struct kobject *fbo_kobj;

/* ----- Directory iteration ----- */

struct fbo_iter_ctx {
    struct dir_context ctx;
    const char        *contains;
    char              *names[FBO_MAX_FILES];
    int                n_names;
};

static bool fbo_filldir(struct dir_context *ctx, const char *name, int namlen,
                        loff_t offset, u64 ino, unsigned d_type)
{
    struct fbo_iter_ctx *fc = container_of(ctx, struct fbo_iter_ctx, ctx);
    char *copy;

    if (fc->n_names >= FBO_MAX_FILES) return false;
    if (namlen >= FBO_MAX_NAME) return true;
    if (namlen == 1 && name[0] == '.') return true;
    if (namlen == 2 && name[0] == '.' && name[1] == '.') return true;

    copy = kmalloc(namlen + 1, GFP_KERNEL);
    if (!copy) return false;
    memcpy(copy, name, namlen);
    copy[namlen] = '\0';

    if (fc->contains && !strstr(copy, fc->contains)) {
```

```
            kfree(copy);
            return true;
        }
        fc->names[fc->n_names++] = copy;
        return true;
    }

    static int fbo_list_dir(const char *path, const char *contains,
                            char **names_out, int max)
    {
        struct path p;
        struct file *filp;
        struct fbo_iter_ctx fc = {
            .ctx.actor = fbo_filldir,
            .contains  = contains,
            .n_names   = 0,
        };
        int ret, i, copy_n;

        ret = kern_path(path, LOOKUP_DIRECTORY, &p);
        if (ret) return ret;

        filp = dentry_open(&p, O_RDONLY | O_DIRECTORY, current_cred());
        path_put(&p);
        if (IS_ERR(filp)) return PTR_ERR(filp);

        ret = iterate_dir(filp, &fc.ctx);
        fput(filp);
        if (ret < 0) {
            for (i = 0; i < fc.n_names; i++) kfree(fc.names[i]);
            return ret;
        }

        copy_n = min(fc.n_names, max);
        for (i = 0; i < copy_n; i++) names_out[i] = fc.names[i];
        for (i = copy_n; i < fc.n_names; i++) kfree(fc.names[i]);
        return copy_n;
    }

    /* ----- Per-file pin sequence ----- */

    /* OEM-specific UFS vendor pin. Replace the printk with your real
     * binding — sysfs write, SCSI passthrough, or vendor ioctl. */
    static int fbo_ufs_pin(const char *path,
                           const struct fiemap_extent *exts, unsigned n, int pin)
    {
        unsigned i;
        pr_info("fbo: ufs_%s %s (%u extent%s)\n",
                pin ? "PIN" : "UNPIN", path, n, n == 1 ? "" : "s");
        for (i = 0; i < n; i++) {
            pr_info("fbo:   lba=0x%llx len=0x%llx flags=0x%x\n",
                    (unsigned long long)exts[i].fe_physical,
                    (unsigned long long)exts[i].fe_length,
                    exts[i].fe_flags);
        }
        return 0;
    }

    static int fbo_pin_one(const char *path)
    {
        struct file *filp;
        struct inode *inode;
        struct fiemap_extent_info fei = { 0 };
        struct fiemap_extent *exts;
        int ret;

        filp = filp_open(path, O_RDONLY, 0);
        if (IS_ERR(filp)) {
            pr_warn("fbo: open %s: %ld\n", path, PTR_ERR(filp));
            return PTR_ERR(filp);
        }
```

```
    inode = file_inode(filp);

    exts = kcalloc(FBO_MAX_EXT, sizeof(*exts), GFP_KERNEL);
    if (!exts) { filp_close(filp, NULL); return -ENOMEM; }

    fei.fi_flags         = FIEMAP_FLAG_SYNC;
    fei.fi_extents_max   = FBO_MAX_EXT;
    fei.fi_extents_start = (struct fiemap_extent __user *)exts;

    if (!inode->i_op->fiemap) {
        ret = -EOPNOTSUPP;
        goto out;
    }
    ret = inode->i_op->fiemap(inode, &fei, 0, ~0ULL);
    if (ret < 0) goto out;

    /* F2FS pin step would go here once exported. */

    ret = fbo_ufs_pin(path, exts, fei.fi_extents_mapped, 1);

out:
    kfree(exts);
    filp_close(filp, NULL);
    return ret;
}

/* ----- Package -> install dir resolution ----- */

static int fbo_find_install_dir(const char *pkg, char *out, size_t outlen)
{
    char *outer[FBO_MAX_FILES];
    char *inner[FBO_MAX_FILES];
    int n_outer, n_inner, i, j, ret = -ENOENT;
    char sub[FBO_MAX_PATH];

    n_outer = fbo_list_dir(FBO_DATA_APP, NULL, outer, FBO_MAX_FILES);
    if (n_outer < 0) return n_outer;

    for (i = 0; i < n_outer; i++) {
        snprintf(sub, sizeof(sub), "%s/%s", FBO_DATA_APP, outer[i]);
        n_inner = fbo_list_dir(sub, pkg, inner, FBO_MAX_FILES);
        if (n_inner > 0) {
            snprintf(out, outlen, "%s/%s", sub, inner[0]);
            for (j = 0; j < n_inner; j++) kfree(inner[j]);
            ret = 0;
            break;
        }
    }
    for (i = 0; i < n_outer; i++) kfree(outer[i]);
    return ret;
}

/* ----- Top-level package pin ----- */

static int fbo_pin_package(const char *pkg)
{
    char install_dir[FBO_MAX_PATH];
    char *files[FBO_MAX_FILES];
    char sub[FBO_MAX_PATH];
    char path[FBO_MAX_PATH];
    int n, i, total = 0;

    if (fbo_find_install_dir(pkg, install_dir, sizeof(install_dir)) < 0) {
        pr_warn("fbo: install dir not found for %s\n", pkg);
        return -ENOENT;
    }
    pr_info("fbo: install dir = %s\n", install_dir);

    /* base.apk + splits */
    n = fbo_list_dir(install_dir, ".apk", files, FBO_MAX_FILES);
    for (i = 0; i < n; i++) {
```

```
            snprintf(path, sizeof(path), "%s/%s", install_dir, files[i]);
            if (fbo_pin_one(path) == 0) total++;
            kfree(files[i]);
        }

        /* oat/<isa>/ */
        snprintf(sub, sizeof(sub), "%s/oat/%s", install_dir, FBO_ISA);
        n = fbo_list_dir(sub, NULL, files, FBO_MAX_FILES);
        for (i = 0; i < n; i++) {
            snprintf(path, sizeof(path), "%s/%s", sub, files[i]);
            if (fbo_pin_one(path) == 0) total++;
            kfree(files[i]);
        }

        /* lib/<isa>/ (may not exist) */
        snprintf(sub, sizeof(sub), "%s/lib/%s", install_dir, FBO_ISA);
        n = fbo_list_dir(sub, NULL, files, FBO_MAX_FILES);
        for (i = 0; i < n; i++) {
            snprintf(path, sizeof(path), "%s/%s", sub, files[i]);
            if (fbo_pin_one(path) == 0) total++;
            kfree(files[i]);
        }

        pr_info("fbo: pinned %d file(s) for %s\n", total, pkg);
        return 0;
    }

    /* ----- sysfs interface ----- */

    static ssize_t pin_store(struct kobject *kobj, struct kobj_attribute *attr,
                             const char *buf, size_t count)
    {
        char pkg[FBO_MAX_NAME];
        size_t n;
        int ret;

        n = min(count, sizeof(pkg) - 1);
        memcpy(pkg, buf, n);
        pkg[n] = '\0';
        while (n > 0 && (pkg[n-1] == '\n' || pkg[n-1] == '\r'))
            pkg[--n] = '\0';
        if (n == 0) return -EINVAL;

        pr_info("fbo: pin request: %s\n", pkg);
        ret = fbo_pin_package(pkg);
        if (ret < 0) return ret;
        return count;
    }

    static struct kobj_attribute pin_attr = __ATTR(pin, 0220, NULL, pin_store);

    /* ----- Module init / exit ----- */

    static int __init fbo_init(void)
    {
        int ret;
        fbo_kobj = kobject_create_and_add("fbo", kernel_kobj);
        if (!fbo_kobj) return -ENOMEM;
        ret = sysfs_create_file(fbo_kobj, &pin_attr.attr);
        if (ret) { kobject_put(fbo_kobj); return ret; }
        pr_info("fbo: loaded (ISA=%s); echo <pkg> > /sys/kernel/fbo/pin\n", FBO_ISA);
        return 0;
    }

    static void __exit fbo_exit(void)
    {
        sysfs_remove_file(fbo_kobj, &pin_attr.attr);
        kobject_put(fbo_kobj);
        pr_info("fbo: unloaded\n");
    }
```

```
        module_init(fbo_init);
        module_exit(fbo_exit);
        MODULE_LICENSE("GPL");
        MODULE_AUTHOR("FBO 3.0");
        MODULE_DESCRIPTION("In-kernel package file enumeration and pin");
```

15.5 How the enumeration works inside the kernel

The flow when userspace writes a package name to `/sys/kernel/fbo/pin`:

```
write(fd, "com.antutu.ABenchMark\n", 22)
    -> sysfs write path
    -> pin_store() callback in fbo_module.ko
        -> trim newline, validate non-empty
        -> fbo_pin_package("com.antutu.ABenchMark")
            -> fbo_find_install_dir():
                -> fbo_list_dir("/data/app/", NULL)
                    -> kern_path()                // resolve "/data/app/"
                    -> dentry_open()              // get a struct file
                    -> iterate_dir()              // calls fbo_filldir
                        -> each H1 entry collected into ctx.names[]
                -> for each H1 hash directory:
                    -> fbo_list_dir(/data/app/H1/, "com.antutu...")
                        -> iterate_dir() with substring filter
                        -> matches "com.antutu.ABenchMark-H2"
                    -> install dir found, break
            -> for the install dir:
                -> fbo_list_dir(install_dir, ".apk")
                    -> matches base.apk, split_config.*.apk
                    -> for each:
                        -> fbo_pin_one(absolute path)
                            -> filp_open()         // open file
                            -> inode->i_op->fiemap // extent map
                            -> fbo_ufs_pin()       // UFS vendor pin (stub)
                            -> filp_close()
                -> fbo_list_dir(install_dir/oat/arm64/, NULL)
                    -> matches base.odex, base.vdex, base.art
                    -> for each: fbo_pin_one()
                -> fbo_list_dir(install_dir/lib/arm64/, NULL)
                    -> empty for modern apps; no-op
            -> pr_info("pinned N file(s) for ...")
```

Every step happens in kernel context. No userspace round trips, no ioctl dispatches across the boundary. The only userspace involvement is the one `write()` to sysfs.

The `iterate_dir()` callback (`fbo_filldir`) is invoked by the filesystem once per directory entry. Inside the callback we filter by an optional substring (used to find the `com.antutu...` child of a hash directory) and copy matching names into the context's array. This is the same mechanism `ls` uses underneath; we are doing it directly without going through the libc dirent wrappers.

15.6 Build system

Two layouts. Both work; pick based on whether you're patching the OEM kernel source tree or shipping as an out-of-tree module.

Out-of-tree (development)

Directory `fbo_module/` containing `fbo_module.c` plus a `Kbuild`:

```
# fbo_module/Kbuild
obj-m := fbo_module.o
```

Build:

```
        KDIR=/path/to/android-kernel-build
```

```
        make -C "$KDIR" M=$(pwd) ARCH=arm64 \
             CROSS_COMPILE=aarch64-linux-android- modules

        # produces fbo_module.ko
```

In-tree (shipping)

Drop the source under `drivers/misc/fbo/` in the kernel tree.

```
# drivers/misc/fbo/Kconfig
config FBO_PIN
    tristate "FBO 3.0 in-kernel package pin"
    default n
    help
      Sysfs-triggered enumeration and UFS pin of an Android
      package's static distribution files.

# drivers/misc/fbo/Kbuild
obj-$(CONFIG_FBO_PIN) += fbo_module.o
```

Then in `drivers/misc/Kconfig`:

```
source "drivers/misc/fbo/Kconfig"
```

And in `drivers/misc/Makefile`:

```
obj-$(CONFIG_FBO_PIN) += fbo/
```

Build the kernel with `CONFIG_FBO_PIN=m` (loadable module) or `CONFIG_FBO_PIN=y` (built into the kernel image).

15.7 Load, test, unload

```
        # Push and load
        adb push fbo_module.ko /data/local/tmp/
        adb shell su -c 'insmod /data/local/tmp/fbo_module.ko'
        adb shell 'dmesg | tail -5'
        # Expect: fbo: loaded (ISA=arm64); echo <pkg> > /sys/kernel/fbo/pin

        # Trigger
        adb shell su -c 'echo com.antutu.ABenchMark > /sys/kernel/fbo/pin'

        # Inspect logs
        adb shell 'dmesg | grep "^fbo:"' | tail -50
        # Expect:
        #   fbo: pin request: com.antutu.ABenchMark
        #   fbo: install dir = /data/app/.../com.antutu.ABenchMark-.../
        #   fbo: ufs_PIN /data/app/.../base.apk (N extents)
        #   fbo:   lba=0x... len=0x... flags=0x0
        #   ...
        #   fbo: pinned 8 file(s) for com.antutu.ABenchMark

        # Unload
        adb shell su -c 'rmmod fbo_module'
```

The sysfs entry has mode `0220` (write-only, owner+group), preventing arbitrary processes from reading it. Production deployments should restrict who can write to it via SELinux policy.

15.8 The three things to wire before production

The skeleton above compiles, loads, and walks the directory tree. It does **not** yet produce real pin effects on the device. Three substitutions complete it:

1. FIEMAP user-pointer issue

The generic FIEMAP path declares its output buffer with the `__user` annotation. On modern kernels (≥5.10) where `set_fs()` has been removed, passing a kernel-space pointer through that path requires either a small patch to skip the `access_ok()` check when called from kernel context, or use of a filesystem-specific extent helper.

The cleaner approach for F2FS is to call `f2fs_map_blocks()` directly:

```
#include <linux/f2fs_fs.h>  /* or the internal header */

struct f2fs_map_blocks map = { 0 };
for (offset = 0; offset < i_size_read(inode); offset += chunk) {
    map.m_lblk = offset >> PAGE_SHIFT;
    map.m_len  = chunk >> PAGE_SHIFT;
    f2fs_map_blocks(inode, &map, F2FS_GET_BLOCK_FIEMAP);
    /* map.m_pblk now contains the physical block */
}
```

This bypasses the generic FIEMAP layer entirely and gives you raw physical block numbers, which is what the UFS pin command wants anyway.

2. F2FS pin export

Without a kernel-callable F2FS pin function, the module cannot protect the file's blocks against F2FS GC. UFS-level pins become stale as GC relocates the underlying data.

The fix is a ~3-line patch to F2FS:

```
/* In fs/f2fs/file.c, after the existing pin implementation: */
int f2fs_pin_file_kernel(struct file *filp)
{
    /* existing pin logic, factored out from f2fs_ioc_set_pin_file */
}
EXPORT_SYMBOL_GPL(f2fs_pin_file_kernel);
```

The module then calls `f2fs_pin_file_kernel(filp)` after `filp_open` and before `fiemap`.

This is the one F2FS-specific change required, and it is a clean API export — not the Android-aware coupling rejected in Section 15.2.

3. UFS vendor pin

Replace the `printk()` calls in `fbo_ufs_pin()` with the actual call to the OEM's pin interface. Three common forms:

```
/* Form A: sysfs node */
struct file *sf = filp_open("/sys/.../ufs_pin", O_WRONLY, 0);
char buf[64];
int len = snprintf(buf, sizeof(buf), "pin %llx %llx\n", lba, length);
kernel_write(sf, buf, len, &sf->f_pos);
filp_close(sf, NULL);

/* Form B: vendor ioctl on a char device */
struct file *cf = filp_open("/dev/fbo_ctl", O_RDWR, 0);
struct fbo_pin_req req = { .lba = lba, .len = length };
vfs_ioctl(cf, FBO_IOC_PIN, (unsigned long)&req);
filp_close(cf, NULL);

/* Form C: direct call into UFS driver via an exported symbol */
ufs_vendor_pin_lba_range(lba, length);   /* defined by vendor driver */
```

Form C is the cleanest if your UFS driver source is in the same build — it's just a function call, no string formatting, no file open. The vendor team that owns the UFS driver typically prefers this once the feature is past prototype.

15.9 Comparison with the userspace tool

Concern	Userspace `fbo_pin`	Kernel `fbo_module.ko`
Lines of code	~600 (full C source)	~250 (skeleton); ~400–500 finished
Build dependencies	gcc or NDK clang	Kernel source tree for target device
Deployment	Push binary + service definition	Insert module or build into kernel image
Triggering	CLI invocation / daemon RPC	`echo <pkg> > /sys/kernel/fbo/pin`
Iteration speed	seconds (rebuild, push)	minutes (rebuild kernel, reflash)
Debugging	gdb, strace, printf	printk + dmesg, kgdb
SELinux fight	Real concern, policy work	None — kernel owns the namespace
F2FS pin	Free via existing per-file ioctl	Requires the small export patch
Lifecycle integration	init.rc, restart policy, SELinux	Module stays loaded, no daemon
Runs on non-Android Linux	Yes (for `pin` subcommand)	No (requires `/data/app/` layout)

The honest deployment posture: build both. The userspace tool is what you use for daily development — measuring `am start -W`, debugging which files are actually pinned, comparing static vs runtime discovery. The kernel module is the production deliverable that goes into the BSP and ships to OEM customers.

They share the same conceptual structure — directory walk, file open, FIEMAP, vendor pin — so anyone who understands one can read the other.

15.10 What this section adds to the deliverable set

File	Lives in	Required?
`fbo_module.c`	source tree	Yes, for kernel-side deployment
`Kbuild`	source tree (alongside `.c`)	Yes
`Kconfig`	only for in-tree build	Optional
Small F2FS pin export patch	OEM kernel `fs/f2fs/file.c`	Required for GC-stable pins
`fbo_module.ko`	build output	Yes — the deployable artifact

`fbo_module.ko` is the singular kernel-side deliverable. Everything else in this document — the userspace tool, the shell helpers, the analysis scripts — supports development and validation around it.